

P2527R3

`std::variant_alternative_index` and `std::tuple_element_index`

Published Proposal, 2023-12-20

This version:

<https://wg21.link/P2527R3>

Author:

[Alex Christensen](#) (Apple)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:

<https://github.com/achristensen07/papers/blob/main/source/P2527r3.bs>

Abstract

A description of `std::variant_alternative_index`, `std::variant_alternative_index_v`, `std::tuple_element_index`, and `std::tuple_element_index_v` which polish use of `std::variant` and `std::tuple` in C++.

Table of Contents

- 1 Introduction
- 2 Revision History
- 3 Questions for LEWG from LWG
- 4 Motivating Use Case: Migrating C code to use `std::variant`

5 Motivating Use Case: Simple object serialization

6 Motivation For `std::tuple_element_index_v`

7 Ill-formed use examples

8 Proposed Wording

Appendix A: possible implementation

§ 1. Introduction

`std::variant` and `std::tuple` seem have a missing piece. `std::get` and `std::get_if` can be used with an index or with a type. There is already a way to get a type from an index (`std::variant_alternative` and `std::tuple_element`) but there is no way to get an index from a type. This adds such a mechanism.

§ 2. Revision History

- r0 Initial version, discussed in the C++ Library Evolution Working Group email list June 27, 2022 through July 13, 2022. This version of the paper was also incorrectly dated 2021-01-18 instead of 2022-01-18.
- r1 Responded to LEWG email feedback by adding `std::tuple_element_index_v`, inheriting from `std::integral_constant`, clarifying what happens with structs and classes that inherit from `std::tuple` and `std::variant`, clarifying what happens with const template parameters, adding spec wording, and possible implementation.
- r2 As requested by LEWG, did some wordsmithing to refine the language of the spec addition with help from Daniel Krügler.
- r3 Responded to feedback from LWG discussing this in Kona on 9 Nov 2023 by removing the const versions. Added questions for LEWG. Updated "possible implementation" section. Replaced "(21.3.2)" with "[meta.rqmts]" in wording additions.

§ 3. Questions for LEWG from LWG

1. Currently proposed wording uses "Mandates" which makes it a compiler error to have a type that is not present in the list of types exactly once. This matches `std::get` but it is not the only possible failure mode. We could change it to "Constraints" which would make it more SFINAE-friendly, which could match a possible future `std::get` after a change like [P0825](#). We could also change the failure mode to do something like returning -1, `std::tuple_size` or `std::variant_size`, or another out-of-bounds value if the type is not found or found multiple times. What should we do? A. Keep "Mandates", B. Switch to "Constraints", or C. Switch to out-of-bounds integers as failure modes
2. The current proposed addition of helpers to `std::variant` and `std::tuple` makes `std::tuple` look more like `std::variant` and less like the other tuple-like types (`std::complex`, `std::array`, `std::pair`, and `std::ranges::subrange`). This does not follow the existing pattern established by `std::tuple_size` and `std::tuple_element` of being able to write generic templates that handle any tuple-like type. What should we do? A. Add `std::tuple_element_index` for the other tuple-like types, B. Remove `std::tuple_element_index` entirely, or C. Keep the existing proposal of only adding to `std::variant` and `std::tuple`

§ 4. Motivating Use Case: Migrating C code to use `std::variant`

Consider the following C program with a unit test:

```
#include <assert.h>

struct NumberStorage {
    enum Type { TYPE_INT, TYPE_DOUBLE } type;
    union {
        int i;
        double d;
    };
};

struct NumberStorage packageInteger(int i) {
    struct NumberStorage packaged;
    packaged.type = TYPE_INT;
    packaged.i = i;
    return packaged;
}
```

```
int main() {
    struct NumberStorage i = packageInteger(5);
    assert(i.type == TYPE_INT);
}
```

In order to migrate it from using a `union` to using a `std::variant` one of the cleanest solutions looks something like this:

```
#include <assert.h>
#include <variant>

// Dear future developers: VariantIndex and NumberStorage must stay in sync
// If you reorder or add to one, you must do the same to the other.
enum class VariantIndex : std::size_t { Int, Double };
using NumberStorage = std::variant<int, double>;

NumberStorage packageInteger(int i) {
    return { i };
}

int main() {
    auto i = packageInteger(5);
    auto expectedIndex = static_cast<std::size_t>(VariantIndex::Int);
    assert(i.index() == expectedIndex);
}
```

Instead, using `std::variant_alternative_index_v` would make the code look cleaner and be easier to maintain:

```
#include <assert.h>
#include <variant>

using NumberStorage = std::variant<int, double>;

NumberStorage packageInteger(int i) {
    return { i };
}
```

```
int main() {
    auto i = packageInteger(5);
    auto expectedIndex = std::variant_alternative_index_v<int, NumberStorage>;
    assert(i.index() == expectedIndex);
}
```

§ 5. Motivating Use Case: Simple object serialization

Another place where `std::variant_alternative_index_v` is useful is when we have an index from a source such as deserialization and we want to decide what type it represents without using any magic numbers:

```
struct Reset { };
struct Close { };
struct RunCommand { std::string command; };

using Action = std::variant<Reset, Close, RunCommand>;

void serializeAction(const Action& action, std::vector<uint8_t>& buffer)
{
    buffer.push_back(action.index());
    if (auto* runCommand = std::get_if<RunCommand>(&action))
        serializeString(runCommand->command, buffer);
}

std::optional<Action> deserializeAction(std::span<const uint8_t> source)
{
    if (!source.size())
        return std::nullopt;

    switch (source[0]) {
    case std::variant_alternative_index_v<Reset, Action>:
        return Reset { };
    case std::variant_alternative_index_v<Close, Action>:
        return Close { };
    case std::variant_alternative_index_v<RunCommand, Action>:
        return RunCommand { deserializeString(source.subspan(1)) };
    }
}
```

```
    return std::nullopt;
}
```

Like the other motivating use case, this could be done with an enum class or `#define RESET_INDEX 0` etc., but this is nicer and references the variant instead of requiring parallel metadata. This was the use case that motivated an implementation in [WebKit](#).

§ 6. Motivation For `std::tuple_element_index_v`

Feedback from r0 indicated that while adding `std::variant_alternative_index_v` for variants, it would be symmetric to add `std::tuple_element_index_v` for tuples.

§ 7. Ill-formed use examples

Like the versions of `std::get` that take a type, the program must be ill formed if the type is not a unique element in the types of the `std::variant` or `std::tuple` such as in these four cases:

```
using Example1 = std::variant<int, double, double>;
auto ambiguous1 = std::variant_alternative_index_v<double, Example1>;

using Example2 = std::variant<int, double>;
auto missing2 = std::variant_alternative_index_v<float, Example2>;

using Example3 = std::tuple<int, double, double>;
auto ambiguous3 = std::tuple_element_index_v<double, Example3>;

using Example4 = std::tuple<int, double>;
auto missing4 = std::tuple_element_index_v<float, Example4>;
```

If the constness of the searched-for type does not match the constness of the type in the `std::variant` or `std::tuple` then there should be a compiler error. This is also the case with `std::get`.

```
using Example5 = std::variant<int, double>;
auto constDoesNotMatch5 = std::variant_alternative_index_v<const int, Exampl
```

```
using Example6 = std::tuple<int, double>;
auto constDoesNotMatch6 = std::tuple_element_index_v<const int, Example6>;
```

Like `std::variant_size` and `std::tuple_size` there should be a compiler error if `std::variant_alternative_index` or `std::tuple_element_index_v` are used with classes or structs that are not `std::variant`s or `std::tuple`s, respectively, including classes or structs that inherit from `std::variant` or `std::tuple`.

```
class Example7 : public std::variant<int, double> { };
auto nonVariantParameter7 = std::variant_alternative_index_v<int, Example7>;
auto nonVariantParameter8 = std::variant_alternative_index_v<int, Example4>;

class Example9 : public std::tuple<int, double> { };
auto nonTupleParameter9 = std::tuple_element_index_v<int, Example9>;
auto nonTupleParameter10 = std::tuple_element_index_v<int, Example2>;
```

If the type of `std::variant` or `std::tuple` is `const`, then there should be a compiler error. This is not the case with `std::variant_size` or `std::tuple_size`.

```
using Example10 = std::variant<int, double>;
auto variantCantBeConst = std::variant_alternative_index_v<int, const Example10>;

using Example11 = std::tuple<int, double>;
auto tupleCantBeConst = std::tuple_element_index_v<int, const Example11>;
```

§ 8. Proposed Wording

These changes are based on the [Working Draft, Standard for Programming Language C++](#) from 2022-12-18

Modify **Header** `<tuple>` synopsis [`tuple.syn`] as follows:

```

[...]
// 22.4.7, tuple helper classes
template<class T> struct tuple_size;    // not defined

template<class... Types> struct tuple_size<tuple<Types...>>;

template<size_t I, class T> struct tuple_element;    // not defined

template<size_t I, class... Types>
    struct tuple_element<I, tuple<Types...>>;

template<size_t I, class T>
    using tuple_element_t = typename tuple_element<I, T>::type;

template<class T, class Tuple> struct tuple_element_index;    // not
defined

template<class T, class... Types> struct tuple_element_index<T,
tuple<Types...>>;

template<class T, class Tuple> constexpr size_t tuple_element_index_v
= tuple_element_index<T, Tuple>::value;

// 22.4.8, element access
[...]
```

Modify **Tuple helper classes** [`tuple.helper`] as follows:

[...]

```
template<size_t I, class T> struct tuple_element<I, const T>;
```

Let TE denote `tuple_element_t<I, T>` of the cv-unqualified type T. Then each specialization of the template meets the *Cpp17TransformationTrait* requirements (21.3.2) with a member typedef type that names the type `add_const_t<TE>`.

In addition to being available via inclusion of the `<tuple>` header, the template is available when any of the headers `<array>` (24.3.2), `<ranges>` (26.2), or `<utility>` (22.2.1) are included.

```
template<class T, class Tuple> struct tuple_element_index;
```

All specializations of `tuple_element_index` meet the *Cpp17BinaryTypeTrait* requirements [meta.rqmts] with a base characteristic of `integral_constant<size_t, N>` for some N.

```
template<class T, class... Types> struct tuple_element_index<T,  
tuple<Types...>>
```

```
    : integral_constant<size_t, N> { _ };
```

Mandates: The type T occurs exactly once in Types.

N is the zero-based index of T in Types.

Modify **Header** `<variant>` synopsis [variant.syn] as follows:

```

#include <compare>    // see 17.11.1
namespace std {
// 22.6.3, class template variant
template<class... Types>
    class variant;

// 22.6.4, variant helper classes
template<class T> struct variant_size;    // not defined
template<class T> struct variant_size<const T>;
template<class T>
    inline constexpr size_t variant_size_v = variant_size<T>::value;

template<class... Types>
    struct variant_size<variant<Types...>>;

template<size_t I, class T> struct variant_alternative;    // not defined
template<size_t I, class T> struct variant_alternative<I, const T>;
template<size_t I, class T>
    using variant_alternative_t = typename variant_alternative<I,
T>::type;

template<size_t I, class... Types>
    struct variant_alternative<I, variant<Types...>>;

template<class T, class Variant> struct variant_alternative_index;    //
not defined
template<class T, class Variant> constexpr size_t
variant_alternative_index_v
    = variant_alternative_index<T, Variant>::value;

template<class T, class... Types>
    struct variant_alternative_index<T, variant<Types...>>;

inline constexpr size_t variant_npos = -1;
[...]
```

Modify **variant helper classes** [**variant.helper**] as follows:

[...]

variant_alternative<I, variant<Types...>>::type

Mandates: $I < \text{sizeof} \dots (\text{Types})$.

Type: The type T_I .

template<class T, class Variant> struct variant_alternative_index;

All specializations of `variant_alternative_index` meet the *Cpp17BinaryTypeTrait* requirements [meta.rqmts] with a base characteristic of `integral_constant<size_t, N>` for some N.

template<class T, class... Types> struct variant_alternative_index<T, variant<Types...>>

: `integral_constant<size_t, N>` { _ };

Mandates: The type T occurs exactly once in Types.

N is the zero-based index of T in Types.

§ Appendix A: possible implementation

```
namespace std {
```

```
namespace detail {
```

```
template<size_t, class, class> struct alternative_index_helper;
```

```
template<size_t index, class Type, class T>
```

```
struct alternative_index_helper<index, Type, variant<T>> {
```

```
    static constexpr size_t count = is_same_v<Type, T>;
```

```
    static constexpr size_t value = index;
```

```
};
```

```
template<size_t index, class Type, class T, class... Types>
```

```
struct alternative_index_helper<index, Type, variant<T, Types...>> {
```

```
    static constexpr size_t count = is_same_v<Type, T> + alternative_index_
```

```
    static constexpr size_t value = is_same_v<Type, T> ? index : alternativ
```

```
};
```

```
template<size_t, class, class> struct tuple_element_helper;
```

```
template<size_t index, class Type, class T>
```

```
struct tuple_element_helper<index, Type, tuple<T>> {
```

```

        static constexpr size_t count = is_same_v<Type, T>;
        static constexpr size_t value = index;
    };

template<size_t index, class Type, class T, class... Types>
struct tuple_element_helper<index, Type, tuple<T, Types...>> {
    static constexpr size_t count = is_same_v<Type, T> + tuple_element_help
    static constexpr size_t value = is_same_v<Type, T> ? index : tuple_elem
};

} // namespace detail

template<class T, class Variant> struct variant_alternative_index;

template<class T, class Variant> struct variant_alternative_index
    : integral_constant<size_t, detail::alternative_index_helper<0, T, Vari
    static_assert(detail::alternative_index_helper<0, T, Variant>::count ==
};

template<class T, class Variant> constexpr size_t variant_alternative_index

template<class T, class Tuple> struct tuple_element_index;

template<class T, class Tuple> struct tuple_element_index
    : integral_constant<size_t, detail::tuple_element_helper<0, T, Tuple>::
    static_assert(detail::tuple_element_helper<0, T, Tuple>::count == 1);
};

template<class T, class Tuple> constexpr size_t tuple_element_index_v = tup

} // namespace std

```